

VoxShield: An E4 Multi-Scale Raw-Waveform CNN Framework for Real-Time Detection of Voice-Cloning Impersonation Attacks

Team YoruH

Smart India Hackathon 2026 · Problem Statement ID: SIH26104

Abstract—Voice-cloning systems can now synthesize convincing speech from a few seconds of reference audio, turning synthetic voices into a practical instrument of impersonation fraud. This paper presents VoxShield, a chunk-based detector that accepts an uploaded or recorded audio file, segments it into 2-second windows, and scores each window with the E4 Multi-Scale Raw-Waveform CNN, a network that operates directly on 16 kHz samples using parallel kernels of size 5, 11 and 21 in each of three blocks. Chunk probabilities are averaged into a file-level score and compared with a decision threshold fixed at 0.44 before system integration. On the current held-out test split, VoxShield attains 77.67% accuracy, 80.80% F1 and an AUC of 0.8859, with 94.00% recall at a 38.67% false-accept rate. Beyond the headline figures we provide a reconstruction of the confusion matrix, bootstrap confidence intervals, a binormal analysis of the operating point, and an account of why a higher, test-tuned F1 (81.08%) was discarded. The system is deployed as an authenticated `/predict` API that returns chunk-level evidence. Weaknesses under background noise and WhatsApp-compressed audio are documented as open research directions rather than omitted.

Index Terms—Voice cloning detection, audio deepfake, raw-waveform CNN, multi-scale convolution, synthetic speech, impersonation attacks, cybersecurity, ROC analysis, binary classification.

I. INTRODUCTION

Text-to-speech and voice-conversion systems have improved to the point where cloning a specific person’s voice no longer requires studio recordings or specialized expertise. A short reference clip, increasingly available from social media, voicemail or a single phone call, can be enough to produce speech that preserves timbre, prosody and speaker identity well enough to pass in casual listening. This matters for cybersecurity because voice has long served as an informal authentication channel: a call from a “known” voice carries an implicit trust that text and email increasingly do not. Cloned-voice impersonation exploits exactly that trust gap, supporting financial fraud, social-engineering attacks on employees, and fabricated evidence, without the visible artifacts that make a doctored image or phishing email easier to flag.

Human listeners are not a reliable detection layer. They judge naturalness and familiarity rather than the statistical regularities that generative models leave behind, and their judgment degrades further on compressed phone or messaging-app audio, where synthesis cues are stripped along with everything else. This is the gap targeted by Problem Statement SIH26104 (AI-Powered Real-Time Detection and Prevention of Voice Cloning Impersonation Attacks) and by VoxShield: an automated, testable classifier that

sits between a suspicious audio file and a human reviewer.

The research gap is not the absence of detectors; WaveFake, RawNet2-style models and the ASVspoof challenges have produced credible ones [1, 2, 3]. The harder problem is *generalization*: detectors trained on one family of generators, one dataset and one recording condition degrade on unseen generators, background noise, messaging-platform compression and short recordings. A strong score on a held-out split of a system’s own training distribution therefore says little about behavior on an unfamiliar caller’s voicemail. VoxShield addresses this by reporting a locked, honestly measured benchmark and by documenting its own failure modes.

In this project, we aim to:

- Build a chunk-based raw-waveform pipeline for REAL/FAKE audio classification
- Design a multi-scale 1-D CNN (E4) that learns directly from samples
- Lock the decision threshold before integration to avoid test-set leakage
- Evaluate with accuracy, precision, recall, F1, MCC and ROC-AUC, with uncertainty
- Expose the model through an authenticated API with chunk-level evidence

The contributions of this paper, all grounded in the implemented system, are: (i) a description of the end-to-end detection pipeline and the E4 architecture at the level confirmed by the team; (ii) the benchmark figures together with a statistical reconstruction (confusion matrix, bootstrap intervals, binormal ROC analysis) that makes the recall/false-accept trade-off quantitatively interpretable; (iii) an explicit separation of what is implemented from what is roadmap; and (iv) a positioning against WaveFake, RawNet2 and ASVspoof 2021 that claims no comparison that was not performed.

II. RELATED WORK

WaveFake [1] provides synthetic speech from several neural vocoders paired with genuine recordings, giving detectors a multi-generator benchmark. Its relevance is methodological: a detector evaluated on a single generator family may learn generator-specific quirks instead of general markers of synthesis, so any raw-waveform detector should ultimately be validated on such data.

RawNet2 [2] applies learned filters directly to the waveform for anti-spoofing, avoiding hand-engineered MFCC or spectrogram front-ends; SincNet [4] is an earlier raw-waveform precedent. VoxShield follows the same philosophy: E4 consumes a mono 16 kHz waveform rather than a precomputed

time-frequency image. Its multi-scale branches echo the parallel-kernel idea of Inception modules [5], transposed to one dimension.

ASVspoof 2021 [3] evaluates spoofing and deepfake detection under channel and compression effects resembling deployment. This is precisely the axis on which VoxShield’s own testing found weakness: WhatsApp-compressed recordings produced a measurable domain shift. Table I summarizes the positioning. The comparison is deliberately descriptive: none of the three systems was run against VoxShield’s data, and E4 has not been scored on their protocols.

III. METHODOLOGY

A. Dataset

The project’s reference material names three data sources: the WaveFake dataset, the ASVspoof 2021 dataset, and an internal collection (“our own datasets”). Sample counts, generator coverage, provenance of the internal collection and the exact training mixture are not documented in the available material and are therefore not asserted. Evaluation uses a held-out test split of the project’s data. Section IV-B shows that the reported rates are exactly consistent with a class-balanced split.

B. Audio Preprocessing

Every incoming file, regardless of container, channel count or sampling rate, undergoes the following steps:

- Decoding via `pydub/fmpeg` and conversion to a **mono** signal
- Resampling to $f_s = 16 \text{ kHz}$ (`torchaudio`)
- Splitting into non-overlapping **2-second chunks** of $L = 32,000$ samples

For a file of duration T seconds the number of full chunks is $K = \lfloor T f_s / L \rfloor$, so variable-length input is handled by scoring fixed-length chunks independently and combining results afterward. No denoising, silence trimming or normalization beyond these steps is confirmed in the project material, and none is claimed. The treatment of a trailing partial chunk is likewise not documented.

C. E4 Multi-Scale Raw-Waveform CNN

Operating on raw samples makes the early layers responsible for learning whatever time- and frequency-domain structure is discriminative, instead of inheriting the biases of a fixed transform. Artifacts of cloned speech, such as phase discontinuities, unnatural regularity in pitch or envelope transitions and vocoder-specific high-frequency patterns, do not always survive hand-designed feature extraction. The confirmed architecture (Fig. 1) is:

- **Stem:** Conv1D $1 \rightarrow 32$ channels, kernel 80, stride 4, followed by batch normalization [6], ReLU and max pooling; the wide kernel gives a large first-layer receptive field suited to raw audio
- **Three multi-scale blocks:** each applies three parallel convolutions with kernels $\{5, 11, 21\}$ to the same input, capturing short-range and long-range temporal structure at once; out-

put widths 64, 128 and a documented 126 channels; each block ends in max pooling

- **Adaptive average pooling** to a fixed-length representation
- **Classifier head:** $256 \rightarrow 128 \rightarrow 1$ with ReLU and dropout 0.35 [7], yielding one logit per chunk

Formally, for block input $\mathbf{x}$ the block output is

$$\mathbf{y} = \text{MP}(\mathcal{M}[\text{Conv}_5(\mathbf{x}), \text{Conv}_{11}(\mathbf{x}), \text{Conv}_{21}(\mathbf{x})]), \quad (1)$$

where $\mathcal{M}$ merges the branches and MP is max pooling. The merge operator (concatenation or summation) and any per-branch normalization are implementation details not recorded in the project material.

Note on Block 3. The documented output width is 126 channels, which breaks the doubling pattern of Blocks 1 and 2 (64, 128). Because the classifier head takes a 256-dimensional input, the value may be a transcription of 256; this has not been verified against the checkpoint and is retained here as recorded. No parameter counts are reported, as none were specified. VoxShield makes no mechanistic claim about which acoustic cues E4 exploits; that would require an interpretability study that has not been performed.

D. Decision Mechanism

Each chunk $\mathbf{x}_k$ yields a logit $z_k = f_\theta(\mathbf{x}_k)$ and a fake probability

$$p_k = \sigma(z_k) = \frac{1}{1 + e^{-z_k}}, \quad k = 1, \dots, K. \quad (2)$$

The file-level score is the mean chunk probability and the label follows from the locked threshold:

$$\bar{p} = \frac{1}{K} \sum_{k=1}^K p_k, \quad \hat{y} = \begin{cases} \text{FAKE}, & \bar{p} \geq \tau \\ \text{REAL}, & \bar{p} < \tau \end{cases} \quad \tau = 0.44. \quad (3)$$

The threshold is “locked”: fixed before, and unchanged during, frontend/backend integration, so that the decision boundary does not drift while other components are built against it. The system returns the label, $\bar{p}$ and the count of flagged versus unflagged chunks. Fig. 2 illustrates how chunk-level evidence coexists with a single file-level decision: a file can contain several chunks below τ yet still be flagged.

E. Application Architecture

E4 is embedded in a deployable service rather than left as a script (Fig. 3). The frontend submits audio as multipart data via HTTP POST to `/predict`. The backend validates the API key *before* any inference, rejecting missing or invalid keys with 401/403, so the expensive step is gated behind a cheap check; the key is held server-side and never embedded in frontend code or the repository. Requests then pass file-presence, format, size and duration validation, preprocessing, inference and aggregation. The confirmed dependencies are PyTorch, torchaudio, pydub and ffmpeg; the specific web and UI frameworks are not documented and are not named.

TABLE I
POSITIONING OF VOXSHIELD RELATIVE TO PRIOR WORK

Work	Core approach	Relevance to VoxShield	Remaining challenge
WaveFake	Multi-generator synthetic-speech dataset	Supplies the cross-generator evaluation VoxShield lacks	No cross-generator number exists; not evaluated on WaveFake
RawNet2	Raw-waveform CNN, learned filters, no hand-crafted features	Direct architectural precedent for E4's raw-waveform design	Head-to-head accuracy unmeasured; no comparative experiment
ASVspoof 2021	Spoofing/deepfake benchmark under channel and codec effects	Matches the compression/noise gap surfaced in VoxShield testing	Not scored on the ASVspoof 2021 protocol
VoxShield	Chunk-based E4 raw-waveform CNN, locked threshold, authenticated API	Working end-to-end pipeline from classifier to deployable service	77.67% acc., 80.80% F1 on own split; not yet validated on the three benchmarks

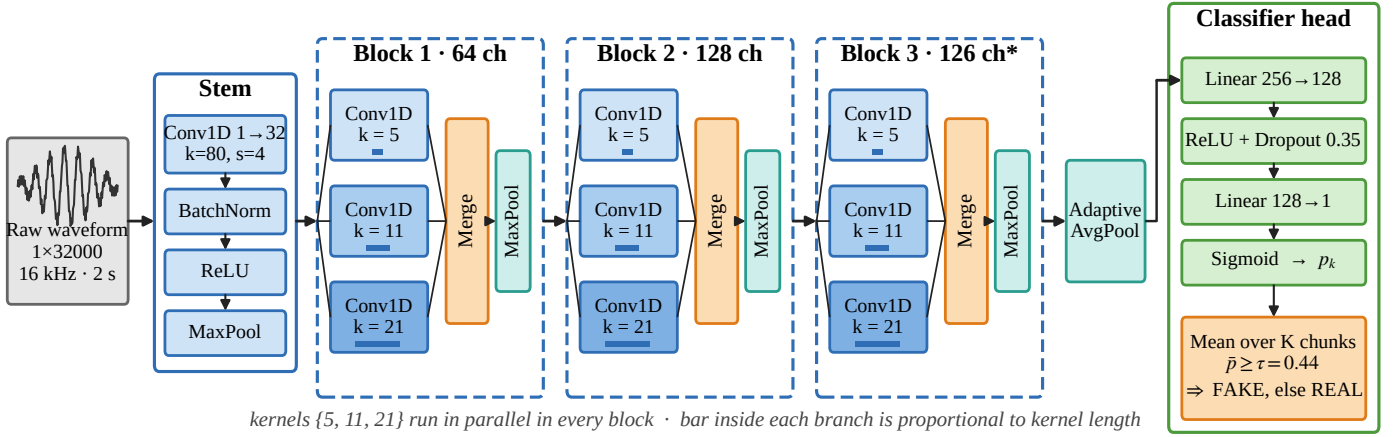


Fig. 1. E4 multi-scale raw-waveform CNN. The stem feeds three multi-scale blocks; each runs kernels 5, 11 and 21 in parallel (blue shade and inner bar indicate kernel size), merges the branches and max-pools. The pooled features pass through a small classifier head to a per-chunk probability, and chunk probabilities are averaged for the file decision. *Block 3 width is reported as documented (126); see text.

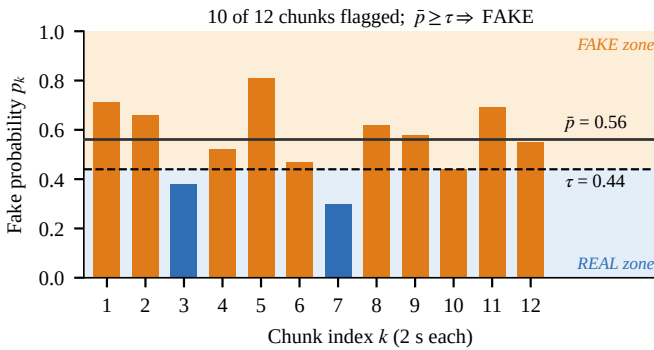


Fig. 2. Illustrative aggregation for a hypothetical 24 s file (values are invented for illustration, not measured outputs). Orange chunks meet $p_k \geq \tau$; the mean $\bar{p}$ decides the file label.

F. Evaluation Protocol

E4 was evaluated on a held-out test split at the locked threshold $\tau = 0.44$. The split ratio, epochs, optimizer, learning-rate schedule, batch size and hardware are not documented and are recorded as unavailable rather than estimated. One methodological decision is confirmed and consequential: an earlier pass swept thresholds and found one giving F1 = 81.08%, but because it was chosen *on the test set* the team declined to report it and instead reports performance at the pre-locked τ . Metrics

follow the usual definitions,

$$\text{Prec} = \frac{TP}{TP+FP}, \quad \text{Rec} = \frac{TP}{TP+FN}, \quad (4)$$

$$\text{FAR} = \frac{FP}{FP+TN}, \quad F_1 = \frac{2TP}{2TP+FP+FN}, \quad (5)$$

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}, \quad (6)$$

with “positive” meaning FAKE, and AUC is the area under the ROC curve [8].

IV. RESULTS

A. Overall Performance

At $\tau = 0.44$ on the current test split, E4 reports the figures in Table II. Accuracy, F1, AUC, recall and FAR are *reported* by the project; precision, specificity and MCC are *derived* from them using the reconstruction in Section IV-B. Intervals are percentile bootstrap intervals [9] (20,000 stratified resamples) conditional on that reconstruction. Fig. 4(a) plots them.

B. Confusion Matrix Reconstruction

The source material gives no cell counts, so we recover them from the reported rates. For a test set with fake prevalence π ,

$$\text{Acc} = \pi \text{Rec} + (1 - \pi)(1 - \text{FAR}). \quad (7)$$

Substituting $\text{Rec} = 0.9400$, $\text{FAR} = 0.3867$ and $\text{Acc} = 0.7767$ gives $\pi = 0.500$: the split is balanced. The smallest balanced

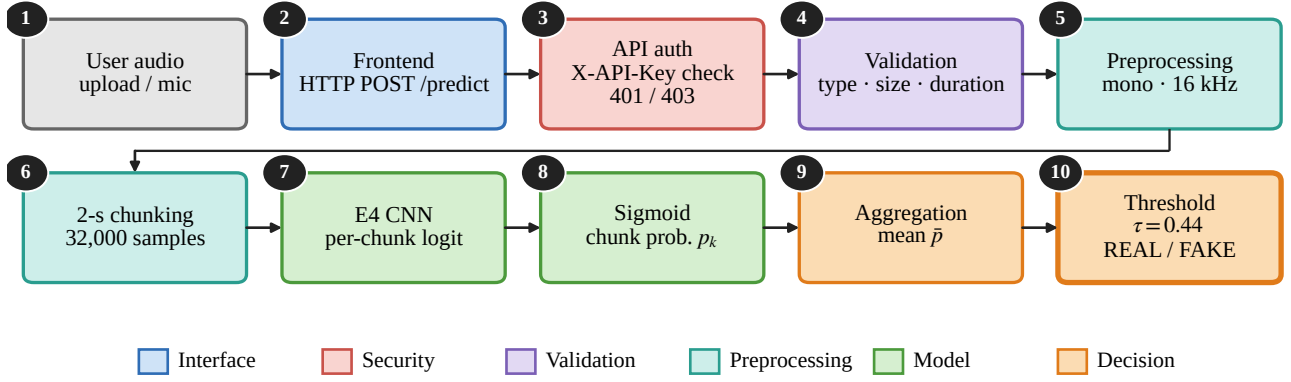


Fig. 3. End-to-end VoxShield workflow in ten steps, colored by functional layer, from audio submission to the rendered REAL/FAKE result with chunk-level evidence.

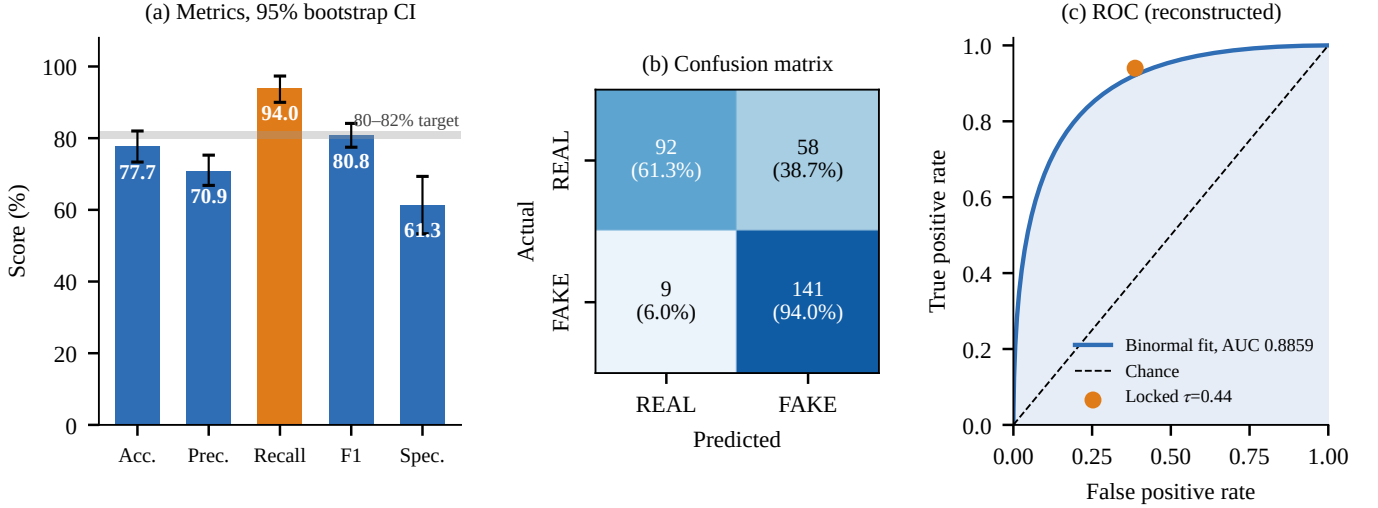


Fig. 4. Results at $\tau = 0.44$. (a) Metrics with 95% bootstrap intervals; the shaded band marks the 80–82% accuracy target. (b) Confusion matrix reconstructed from the reported rates under a balanced 150/150 split (counts and row percentages). (c) ROC reconstruction, a binormal fit to the reported AUC of 0.8859, with the locked operating point; illustrative, as the measured curve was not retained.

TABLE II
E4 PERFORMANCE AT THE LOCKED $\tau = 0.44$

Metric	Value	95% CI	Basis
Accuracy	77.67%	[73.3, 82.0]	reported
F1-score	80.80%	[77.5, 84.1]	reported
ROC-AUC	0.8859	—	reported
Recall (TPR)	94.00%	[90.0, 97.3]	reported
False-accept rate	38.67%	[30.7, 46.7]	reported
Precision	70.85%	[66.8, 75.3]	derived
Specificity (TNR)	61.33%	[53.3, 69.3]	derived
MCC	0.586	[0.50, 0.67]	derived
Youden's J	0.553	—	derived

split reproducing every reported rate exactly is 150 FAKE and 150 REAL files ($TP=141$, $FN=9$, $FP=58$, $TN=92$; accuracy 233/300, FAR 58/150). It also reproduces the reported F1, $282/349 = 80.80\%$, an independent check, and implies precision $141/199 = 70.85\%$. This is a *reconstruction*, not logged data (Fig. 4(b)); any larger consistent test set would only narrow the intervals in Table II, so they are conservative.

C. ROC and AUC Analysis

The ROC curve itself was not retained, so Fig. 4(c) shows an equal-variance binormal model fitted to the reported AUC, for which $AUC = \Phi(d'/\sqrt{2})$ gives $d' = 1.70$. The locked operating point (FAR 0.387, TPR 0.940) lies slightly above the fitted curve, so the fit is a reasonable, though not exact, summary; it should not be read as a measured curve.

The binormal model also lets us interpret $\tau = 0.44$. At that operating point the likelihood ratio of the score density is ≈ 0.38 , and a Bayes-optimal decision at balanced prior would sit there only if a missed fake were about $1/0.38 \approx 2.6$ times as costly as a false alarm. That is a modest cost asymmetry: the threshold favors recall, but not extremely, and a larger asymmetry would justify moving further along the curve. This inference is model-dependent and is offered as an analytical aid, not a measured result.

D. Threshold Integrity

The discarded threshold raised F1 from 80.80% to 81.08%, a gain of 0.28 percentage points. That is about 4% of the width of

TABLE III
STATED ROADMAP (NO RESULTS REPORTED)

Version	Target
Current	Implemented: checkpoint, locked τ , authenticated /predict
V1.1	WhatsApp / messaging-compression domain adaptation
V1.2	Telephone and call-recording robustness
V1.3	Combined V1.1 and V1.2 robustness
V2	Microphones, TTS diversity, cloning methods, noise, codecs

the F1 interval in Table II (≈ 6.6 points), so the gain is statistically indistinguishable from noise, while selecting it on the test set would have contaminated the estimate. Reporting the locked figure is therefore both the more honest and, in effect, the lossless choice.

E. Known Failure Conditions

The team’s feasibility testing identified two conditions with degraded behavior: *background-dominant audio* and *externally WhatsApp-compressed recordings*, where compression-induced domain shift was observed. Neither has a quantified figure; both are recorded as qualitative weaknesses. No result exists for generators absent from the training data, so generalization to unseen cloning methods is an open question rather than a measurement.

V. DISCUSSION

Operating point. For a security tool, a missed fake is usually costlier than a false alarm that a human reviewer can adjudicate, so 94% recall with a 38.67% false-accept rate is a defensible orientation. It is not a resolved trade-off: roughly 29 of every 100 flagged files are genuine (precision 70.85%), which limits usefulness at high call volumes. The team’s practical target of 80–82% accuracy is only 2–4 points above the current value and lies within the reconstructed accuracy interval, so demonstrating improvement will require a larger test set or paired comparison, not merely a higher point estimate.

Generalization. The central open problem, shared with the field, is robustness beyond the training distribution. The roadmap in Table III targets it directly; none of the entries has reported results yet.

Security and deployment. Enforcing API-key authentication before inference limits compute abuse by unauthenticated requests. The material does not confirm rate limiting, audit logging, encryption beyond standard HTTPS, or a retention policy for submitted audio; these are natural next steps for production. Plausible uses include reviewing call recordings in fraud investigations, evidence verification, and first-pass screening before human review, but none is claimed as an existing deployment. The chunk-independent design is structurally amenable to parallel scoring, yet no latency or throughput figures exist, so no real-time performance claim is made beyond the architecture.

Unmeasured limitations. Cross-speaker and multilingual generalization, recordings shorter than one chunk, confidence

calibration of $\bar{p}$, robustness to adversarially crafted audio, streaming inference, and evaluation on WaveFake or the full ASVspoof 2021 protocol all remain unreported, and none should be assumed to hold or fail.

VI. CONCLUSION

VoxShield treats voice-cloning detection as chunk-level audio classification aggregated to a file-level decision, and connects the E4 Multi-Scale Raw-Waveform CNN to a working, authenticated application. On its current test split it reports 77.67% accuracy, 80.80% F1 and 0.8859 AUC, with 94.00% recall and a 38.67% false-accept rate. Our analysis shows the split is consistent with a balanced 150/150 design, that precision is about 70.9%, and that the locked threshold implies a moderate 2.6:1 preference for catching fakes. The decision to discard a test-tuned 81.08% F1 preserves the integrity of the benchmark. Generalization to unseen generators, robustness to compression and noise, and the recall/false-accept balance remain active work. VoxShield’s output should be understood as a model-based risk assessment that supports human review, not as proof of a recording’s authenticity or a fully generalized deepfake detector.

REFERENCES

- [1] J. Frank and L. Schönherr, “WaveFake: A data set to facilitate audio deepfake detection,” in *NeurIPS Datasets and Benchmarks*, 2021, arXiv:2111.02813.
- [2] H. Tak, J. Patino, M. Todisco, A. Nautsch, N. Evans and A. Larcher, “End-to-end anti-spoofing with RawNet2,” in *Proc. IEEE ICASSP*, 2021, arXiv:2011.01108.
- [3] J. Yamagishi *et al.*, “ASVspoof 2021: Accelerating progress in spoofed and deepfake speech detection,” in *Proc. ASVspoof 2021 Workshop*, 2021, arXiv:2109.00537.
- [4] M. Ravanelli and Y. Bengio, “Speaker recognition from raw waveform with SincNet,” in *Proc. IEEE SLT*, 2018.
- [5] C. Szegedy *et al.*, “Going deeper with convolutions,” in *Proc. IEEE CVPR*, 2015.
- [6] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. ICML*, 2015.
- [7] N. Srivastava *et al.*, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.
- [8] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.
- [9] B. Efron, “Bootstrap methods: Another look at the jackknife,” *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.